Growth Rate of the Output of Handshake Theorem

vs

Time Complexity of Handshake Theorem

What does Growth Rate means ?

How fast a value increases as input size `n` becomes large.

Here in the problem we have :

```
int handshake(int n) // Function to calculate the number of
handshakes
{
    if (n == 0 || n == 1) // Base case
    {
        return 0; // If there is only one person in the
room, there are no handshakes
    }
    cout << n << endl;
    return handshake(n - 1) + (n - 1); // Recursive call :
n-1 people shake hands with each other and the remaining
person shakes hands with n-1 people
}
```

We know , $H(0) = 0$ ***and*** $H(1) = 0$ ***hence both are constants i.e.*** `C` ,

we can use ` 1 ***or*** C` ***just to represent constant. Not from Time Complexity but just like Time Complexity*** .

$\boldsymbol{The\ Growth\ Rate\ of\ the\ Program\ is}$:

$$H(n) = H(n-1) + (n-1)$$

$\boldsymbol{And\ Base\ Case\ , when\ H(0)\ or\ H(1) = 0\ in\ the\ program.}$

$\boldsymbol{We\ will\ go\ through\ substitution\ method}$:

$$H(n-1) = H(n-2) + (n-2)$$

$\boldsymbol{Substituting\ H(n-1), in\ H(n) we\ get}$:

$$H(n) = H(n-2) + (n-2) + (n-1)$$

$\boldsymbol{Similarly,}$

$$H(n-2) = H(n-3) + (n-3)$$

$\boldsymbol{Substituting\ H(n-2), in\ H(n) we\ get}$:

$$H(n) = H(n-3) + (n-3) + (n-2) + (n-1)$$

$\boldsymbol{if\ , it\ continue\ `k`\ times\ , then}$:

$$H(n) = H(n-k) + (n-k) + (n-k+1) + \cdots + (n-2) + (n-1)$$

$\boldsymbol{Final\ Step\ (stop\ at\ base\ case)}$

$\boldsymbol{We\ stop\ when}$:

$$\boldsymbol{n - k = 1 \Longrightarrow k = n - 1}$$

$\boldsymbol{So,}$

$$\boldsymbol{H(n) = H\big(n - (n-1)\big) + \big(n - (n-1)\big) + (n - (n-1) + 1) + \cdots + (n-2) + (n-1)}$$

$$\boldsymbol{H(n) = H(n - n + 1) + (n - n + 1) + (n - n + 1 + 1) + \cdots + (n-2) + (n-1)}$$

$$\boldsymbol{H(n) = H(1) + (1) + (2) + \cdots + (n-2) + (n-1)}$$

$\boldsymbol{We\ , know, H(1) = 1\ or\ C\ for\ constant.}$

$$\boldsymbol{H(n) = 1 + (1) + (2) + \cdots + (n-2) + (n-1)}$$

$$\boldsymbol{\therefore H(n) = 1 + \sum_{i=1}^{n-1} i = 1 + \frac{n(n-1)}{2} = \frac{n^2 - n + 2}{2}}$$

$\boldsymbol{Now\ applying\ , Big\ O\ , we\ get}$:

$$= \frac{1}{2} \times O(2 + n^2 - n)$$
$$= O(n^2 - n) + O(2)$$
$$= O(n^2)$$

$\boldsymbol{And\ we\ know\ , f(n) = n^2 = Quadratic\ Growth\ Rate}$, $\boldsymbol{Hence, Handshake\ Problem\ has\ Quadratic\ Growth\ Rate}$.

$\boldsymbol{But\ what\ is\ Time\ Complexity\ of\ the\ program}$:

$$T(n) = T(n-1) + c\ or\ T(n-1) + 1 \approx O(n)$$

Why so?

To put it simply, ***Time Complexity defines the*** Effort, *while* ***Output Value Growth defines the*** Reward. *They measure two entirely different constraints on the computer system. Time complexity dictates how long user will have to wait, while output value growth dictates how fast the result increases.*

$\boldsymbol{Time\ Complexity\ of\ an\ Algorithm\ does\ NOT\ describe\ how\ fast\ the\ output\ value\ grows.}$

$\boldsymbol{It\ describes: How\ the\ number\ of\ operations\ (running\ time) grows\ with\ input\ size\ n.}$

$\boldsymbol{Time\ Complexity}$

- $Measures$ $\boldsymbol{work\ done\ by\ the\ algorithm}$
- $Depends\ on:$
  - $loops$
  - $recursion\ calls$
  - $operations\ executed$

$$\boldsymbol{T(n)}$$

Growth Rate of Output Value

- Measures **how the result itself increases**
- Depends on:
  - the **mathematical function/problem**

$$\boldsymbol{f(n)}$$

Concept	Defines
Time Complexity	How fast the algorithm runs
Growth Rate of Output	How fast the result increases

$\boldsymbol{Where\ , Algorithm = \ Process\ and\ Output = Result.}$